LISTA 2 - Árvore B – 3,0 Pontos

1) Escreva uma função que, dada uma árvore-B e uma chave, retorne o **antecessor imediato (primeiro valor menor que um valor qualquer presente na subárvore da esquerda do valor)** da chave.

2) Para uma árvore B de ordem d > 0 e altura h > 0, calcule em função de d e h:
    a. O número mínimo de chaves que ela pode possuir (nMIN).
    b. O número máximo de chaves que ela pode possuir (nMAX).

3) Escreva uma função que, dada uma árvore-B, realize a contagem de quantos elementos existem na árvore.

4) Escreva uma função que dado um nóB, calcule a altura desse nó. Lembre-se a definição de altura pode ser simplificada para o número de arestas deste nó até o nó folha.

5) Escreva uma função que localize o valor máximo presente na árvore B.

DESAFIO

DESAFIO) Nas remoções em árvores B, há diversos casos a considerar. Em um deles, se uma **folha** tem uma chave removida e o número de chaves fica **abaixo** do mínimo permitido, pode-se proceder uma redistribuição das chaves entre outros procedimentos.

Mínimo de chaves permitido por nó (exceto raiz) = $\lceil m / 2 \rceil - 1$
O símbolo [ ] significa arredondar para cima.

Ao optar por uma redistribuição, observa-se que o nó que teve a chave removida pode receber chaves do seu nó irmão adjacente até alcançar o número mínimo de chaves permitidas em um nó.

Dois nós P e Q são chamados *irmãos adjacentes* se têm o mesmo pai W e são apontados por 2 ponteiros adjacentes em W.

Visualmente, nós irmãos adjacentes são aqueles que estão “ao lado” um do outro na árvore e TEM o mesmo pai.

Considerando especificamente **o caso da chave a ser removida estar em uma folha e o número de chaves ficar abaixo do mínimo permitido,** tome k como a chave a remover e implemente:

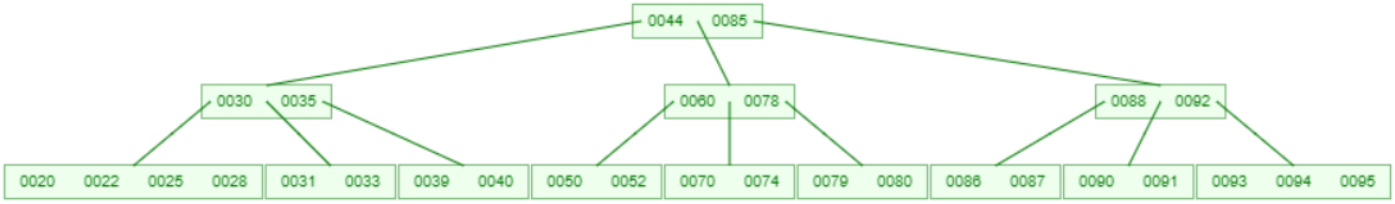
a) uma função que retorne um ponteiro para a folha irmã adjacente da folha onde está a chave k que tenha o maior número de chaves. Se a folha com o maior número de chaves tiver somente o mínimo de chaves (ou seja, não pode perder chaves) a função deve retornar NULL.
b) uma função que dados a chave k a ser removida em uma folha, o ponteiro da folha, o ponteiro da folha irmã adjacente com possibilidade de perder chaves (se ela existir), remova a chave k e reorganize o que for necessário, redistribuindo as chaves restantes.
c) Uma função com o objetivo de remover a chave k, que determine se k está numa folha e que chame as funções anteriores para concluir a remoção.

Considere:

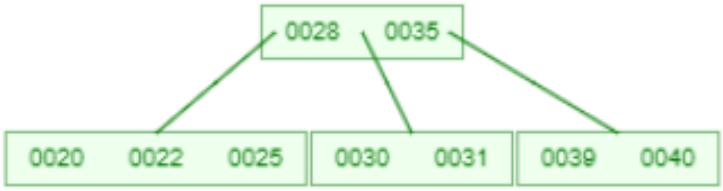
- que esse caso de remoção nunca será feito na raiz (mesmo que ela seja uma folha), ou seja, você não precisa tratar isso nas suas funções.
- os nós irmãos adjacentes de uma folha também são folhas

Exemplos práticos:

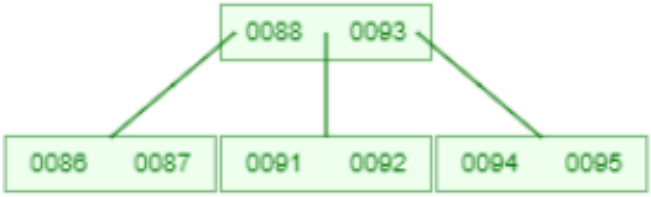
Considere a árvore B de ordem 5 a seguir:



Se o valor a ser removido for o 33, o nó irmão adjacente melhor candidato para perder chaves seria o [20 22 25 28], ou seja, nó irmão do nó [31 33] que tem o maior número de chaves. Resultando na subárvore a seguir:



Se o valor a ser removido for o 90, o nó irmão adjacente melhor candidato para perder chaves seria o [93 94 95], ou seja, nó irmão do nó [90 91] que tem o maior número de chaves. Resultando na subárvore a seguir:



Se o valor a ser removido for o 60, tanto o nó [30 35] quanto o nó [88 92] já estão no limite e não podem perder chaves, logo a função da letra a retorna NULL e a da letra b nem é chamada.

Se o valor a ser removido for o 79, o nó [70 74] será o único candidato, mas já está com a quantidade mínima de chaves, logo a função da letra a retorna NULL e a da letra b nem é chamada.